

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	ATHIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				

Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT3-COMPARATIVE ANALYSIS

OPEN

✓ Completed

Review →

1/1 answered

View Rubric

Criteria	Level 4	Exemplary Level 3	Proficient Level 2	Developing Level 1	Beginning
Selection of Studies/Parameters 20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies	
Comparison & Differentiation 25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison, mostly descriptive	
Critical Evaluation 25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided	
Synthesis & Insight Generation 20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated	
Clarity 10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand	

1. Introduction

An e-learning platform may contain thousands or millions of learning resources such as videos, lectures, notes, quizzes, assignments, and practice problems. Since every student has different interests, learning levels, goals, and interaction patterns, showing the same content to every student is inefficient. A personalized recommendation system can solve this problem by representing relationships between students and learning resources as a graph and using graph-based ranking algorithms to identify the most relevant content.

The main objective is to recommend high-quality and personalized learning content while maintaining computational efficiency and scalability. The system should consider user diversity, large content volume, changing learning behavior, and educational effectiveness.

2. Problem Definition

For every student u , the system must rank a large collection of learning items and recommend the items most likely to improve engagement and learning outcomes. The recommendation should be based on interactions such as viewing, completing, liking, bookmarking, attempting quizzes, and achieving good results.

The recommendation problem can be formulated as: Given a graph $G = (V, E)$, where nodes represent students and content items and edges represent interactions or relationships, calculate a relevance score for each candidate content item and return the top- k items for each student.

3. Graph Representation

A heterogeneous graph is suitable because an educational platform contains different types of entities and relationships.

Node / Edge	Meaning	Example
Student node	Represents a learner	Student S1
Content node	Represents learning material	Python Functions Video
Topic node	Represents a subject/topic	Recursion
Student → Content	Interaction with content	Viewed, completed, attempted
Content → Topic	Content-topic relationship	Video belongs to Recursion
Student → Topic	Learner interest or mastery	High interest in Algorithms

4. Graph-Based Ranking Algorithms

Several graph algorithms can be used to rank educational content. The most useful approaches include Personalized PageRank, random-walk methods, similarity-based ranking, and centrality measures. Personalized PageRank is particularly suitable because it can produce a ranking specific to an individual student.

5. Personalized PageRank Approach

Personalized PageRank starts a random walk from nodes associated with a particular student. At every step, the walker follows graph edges with a probability determined by edge weights. With a smaller probability, it returns to the student's preference or profile nodes. After repeated iterations, each content node receives a score. Higher scores indicate stronger relevance to that student.

A simplified formulation is:

$$\mathbf{PR} = \alpha(\mathbf{P}^T \mathbf{PR}) + (1 - \alpha)\mathbf{q}$$

Here, $\mathbf{P}$ is the normalized transition matrix, α is the damping factor, and $\mathbf{q}$ is the personalized preference vector. The final PageRank values are used to rank candidate content.

6. Optimized Recommendation System Architecture

1. Student Activity Collection → collect views, completion, quiz scores, searches, bookmarks, and feedback.
2. Feature Extraction → calculate interests, topic mastery, difficulty preference, and recent activity.
3. Graph Construction → create student, content, topic, and interaction nodes with weighted edges.
4. Candidate Generation → retrieve a manageable set of relevant content using graph traversal, topic filtering, or similarity.
5. Graph-Based Ranking → apply Personalized PageRank or another graph ranking method.
6. Constraint Filtering → remove completed, unsuitable, duplicate, or prerequisite-incompatible items.
7. Top-k Recommendation → return the highest-ranked content to the student.
8. Feedback Loop → use new interactions and learning results to update scores and improve future recommendations.

7. Recommendation Score

A practical system can combine the graph score with educational and behavioral features. For example, a final ranking score can be defined as:

$$\text{Final Score} = w_1(\text{GraphScore}) + w_2(\text{TopicMatch}) + w_3(\text{MasteryGap}) + w_4(\text{Recency}) + w_5(\text{ContentQuality}) - w_6(\text{Repetition})$$

The weights can be learned or tuned using validation data. This prevents the system from depending only on popularity and allows educational usefulness to influence ranking.

8. Constraints and Challenges

Constraint	Impact
User diversity	Students differ in age, background knowledge, goals, interests, pace, and preferred learning format.
Content volume	Large platforms may contain millions of resources, making full graph ranking expensive.
Cold-start users	A new student has little or no interaction history, so personalization is initially limited.
Cold-start content	New content has few interactions and may be under-recommended.
Dynamic behavior	Student interests and mastery can change over time.
Content quality	Highly clicked content is not necessarily educationally effective.
Prerequisites	Advanced content should not be recommended before necessary foundational concepts.
Popularity bias	Popular resources may dominate rankings even when they are not the best match.
Diversity	The top-k list should avoid recommending many nearly identical resources.
Privacy	Student interaction data must be handled responsibly and securely.

9. Scalability in Large Educational Platforms

A direct application of graph ranking over the complete student-content graph can become expensive when the number of nodes and edges becomes very large. Therefore, the optimized system should avoid recomputing rankings for every possible content item.

Candidate generation: first select a small relevant subset instead of ranking all content.

Sparse graph storage: store only meaningful interactions and relationships.

Caching: cache rankings or candidate lists for frequently active students.

Incremental updates: update affected scores after new interactions rather than rebuilding everything.

Parallel processing: distribute graph computations across multiple machines when necessary.

Approximate ranking: use approximate algorithms when exact ranking provides little practical benefit.

Batch processing: perform expensive global calculations periodically while keeping real-time ranking lightweight.

10. Computational Efficiency

The efficiency of the system depends on the graph size, number of edges, number of candidate items, and number of PageRank iterations. A full PageRank computation may require repeated matrix-vector operations and can be costly on a very large graph.

For a sparse graph, one PageRank iteration is approximately proportional to the number of edges, $O(|E|)$. If k iterations are required, the approximate computation is $O(k|E|)$. Personalized ranking for every student independently can still be expensive, so candidate generation, caching, and approximate or incremental methods are important.

A Greedy ranking alternative can be much cheaper: the system can score nearby or candidate content and select the highest-scoring items. However, a purely greedy strategy may miss content that becomes relevant through multi-step graph relationships.

11. Pseudocode

INPUT: Student u , graph G , number of recommendations k

1. Collect recent interactions of student u .
2. Identify preferred topics and learning gaps.
3. Generate candidate content nodes connected to u .
4. Assign weights to student-content and content-topic edges.
5. Initialize personalized PageRank vector using u 's profile.
6. Repeat until convergence or maximum iterations:
 Update PageRank scores using graph transitions.
7. Calculate final score for every candidate.
8. Remove completed, duplicate, unsuitable, or prerequisite-invalid content.
9. Sort remaining candidates by final score.
10. Return the top- k content items.
11. Record feedback and learning outcomes for future updates.

12. Evaluation Metrics

Metric	Purpose	Educational Interpretation
Precision@K	Measures relevant items among top K	How many recommendations are useful
Recall@K	Measures relevant items retrieved	How much useful content was found
CTR / Engagement	Measures interaction with recommendations	Whether students engage with suggested resources
Completion rate	Measures completed recommended content	Whether recommendations are practical and appropriate
Quiz improvement	Measures change in assessment performance	Direct evidence of learning benefit
Time-to-mastery	Measures time required to reach mastery	Efficiency of learning
Diversity	Measures variety of recommended items	Avoids repetitive recommendations
Coverage	Measures content/user coverage	Prevents the system from recommending only popular items

13. Interpretation in Terms of Learning Effectiveness

High engagement alone does not prove that a recommendation system improves learning. A student may click an entertaining video without gaining mastery. Therefore, the system should evaluate educational outcomes such as quiz performance, completion, concept mastery, retention, and reduction in learning gaps.

For example, if the graph model recommends a recursion exercise because the student has performed poorly on recursion-related assessments, and the student's subsequent quiz score improves, the recommendation has stronger educational evidence than a recommendation based only on popularity.

14. Example Scenario

Assume Student S1 has recently completed lessons on arrays and linked lists, has struggled with recursion, and frequently watches short programming videos. The graph connects S1 to these topics and to content consumed by students with similar learning patterns.

The graph ranking may give high scores to a recursion fundamentals video, a recursion practice quiz, and a visual recursion explanation. Content that requires advanced dynamic programming may receive a lower score because the prerequisite concepts are not yet mastered.

This demonstrates why graph-based ranking is useful: relationships can be discovered through multiple paths rather than relying only on direct clicks.

15. Advantages

Provides personalized recommendations.

Uses relationships among students, topics, and content.

Can discover indirect similarities through graph paths.

Can incorporate learning outcomes and content quality.

Supports diverse recommendation strategies.

Can be optimized using candidate generation and caching.

16. Limitations

Large graphs require significant computational resources.

New users and new content suffer from limited interaction data.

Poor-quality interaction data can produce inaccurate rankings.

Personalized PageRank may require repeated computation.

Popularity and historical bias can be propagated through the graph.

A recommendation that maximizes engagement may not always maximize learning.

17. Critical Evaluation

Graph-based ranking provides a strong foundation for personalized e-learning because it models the platform as a network rather than treating every content item independently. However, the best system should not rely on a single ranking algorithm. A hybrid design can combine graph relevance, prerequisite rules, mastery estimation, content quality, diversity, and recent behavior.

From a DAA perspective, the key optimization is to reduce the amount of computation without significantly reducing recommendation quality. Candidate generation limits the search space, sparse representations reduce memory use, and caching or incremental computation reduces repeated work. These techniques make graph-based recommendation more feasible for large platforms.

18. Complexity Summary

Operation	Approximate Complexity	Purpose
Graph construction	$O(E)$	Create weighted relationships
One sparse PageRank iteration	$O(E)$	Propagate relevance
k PageRank iterations	$O(kE)$	Compute ranking
Greedy candidate ranking	Depends on candidates; typically $O(C \log C)$	Fast ranking of candidate set
Top- k selection	$O(C \log k)$ with a heap	Select best recommendations

19. Conclusion

An optimized e-learning recommendation system can represent students, learning resources, topics, and interactions as a graph and use graph-based ranking to personalize content. Personalized PageRank is especially useful because it ranks resources according to the student's individual context. However, large educational platforms require scalability techniques such as sparse graphs, candidate generation, caching, incremental computation, and parallel processing.

The effectiveness of the system should ultimately be judged by learning outcomes rather than clicks alone. A successful system recommends content that is relevant, diverse, appropriately difficult, prerequisite-aware, and capable of improving mastery. Thus, graph-based ranking offers a useful balance between personalization, computational efficiency, and educational value.